

Pontifícia Universidade Católica de Campinas
Escola Politécnica

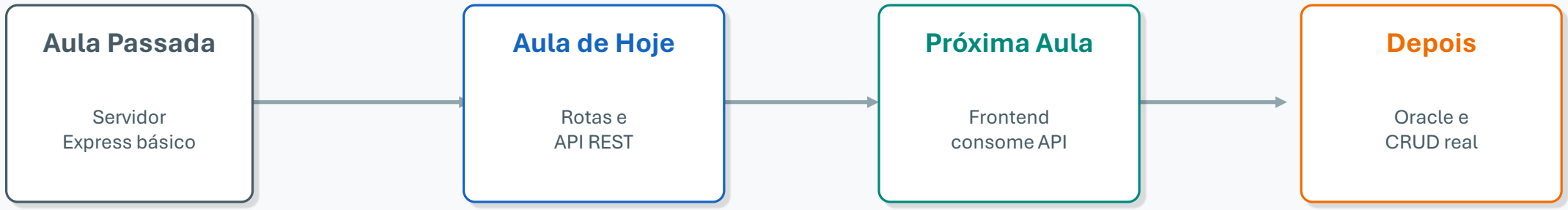
Rotas e API REST

Introdução as Tecnologias WEB

José Guilherme Picolo
E-mail: jose.picolo@puc-campinas.edu.br

Onde esta aula entra no percurso?

A API é a ponte entre a interface e os dados do sistema

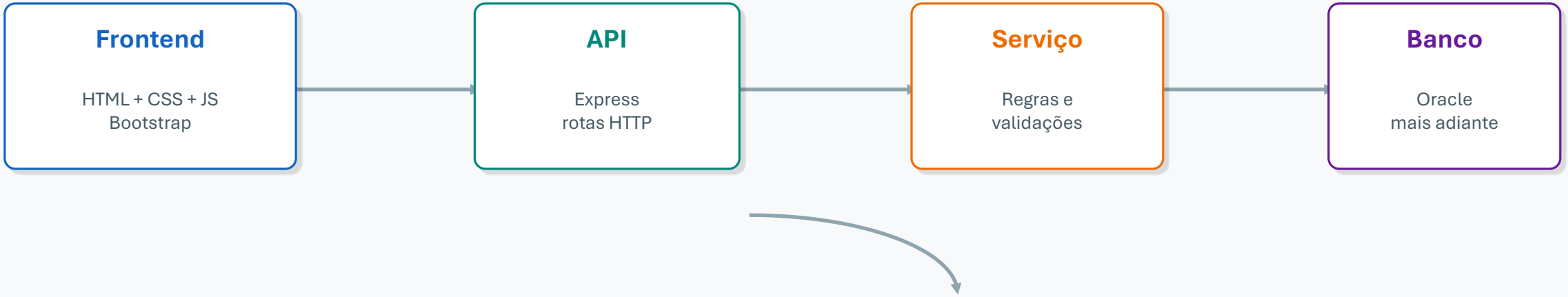


Ideia-chave

Hoje ainda usaremos um array em memória. Ele simula o banco de dados.

A arquitetura que estamos construindo

Da tela do usuário até a resposta em JSON



Nesta aula

O foco fica no bloco da API: rotas, métodos, parâmetros, corpo da requisição e status HTTP.

Na próxima etapa

O frontend usará fetch para conversar com essas rotas.

Retomada: servidor Express da Aula 07

Ponto de partida para transformar o servidor em API

```
import express, { Request, Response } from "express";

const app = express();
const PORT = 3000;

app.get("/", (req: Request, res: Response) => {
  res.send("Servidor funcionando!");
});

app.listen(PORT, () => {
  console.log(`Servidor rodando na porta ${PORT}`);
});
```

O que ele faz?

Responde apenas uma rota simples: GET /

O que falta?

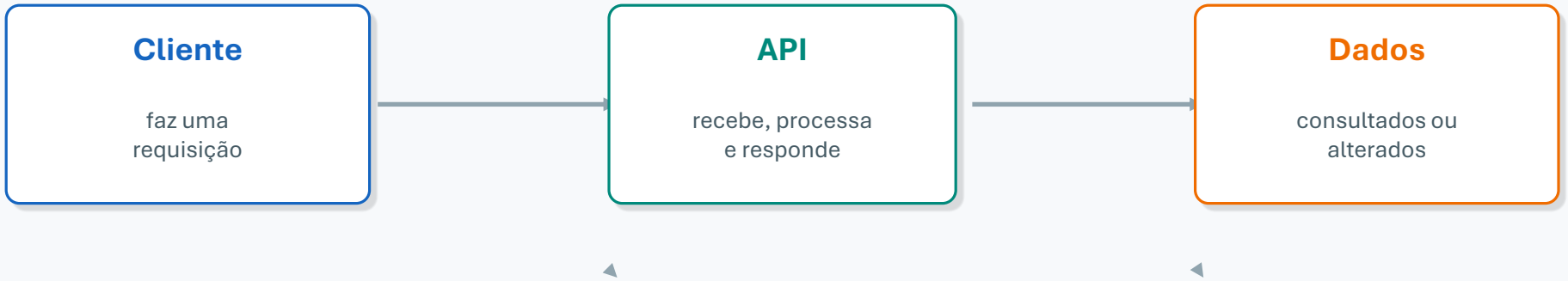
Rotas para recursos reais do sistema.

Próximo passo

Transformar o servidor em uma API.

O que é uma API?

Uma forma padronizada de comunicação entre sistemas



No nosso contexto

A API será o canal pelo qual a interface web enviará comandos para o backend e receberá dados em JSON.

API não é página HTML

O backend devolve dados; quem monta a tela é o frontend

Página HTML

Retorna marcação visual: títulos, parágrafos, botões, tabelas.

```
<h1>Alunos cadastrados</h1>
<table>...</table>
```

API REST

Retorna dados estruturados para outro programa interpretar.

```
[{
  "id": 1,
  "nome": "Ana Clara",
  "curso": "Engenharia de Software",
  "situacao": "Ativo"
}]
```

Frontend transforma JSON em interface.

Recurso: o “substantivo” da API

Uma API REST normalmente organiza rotas em torno de recursos

- Um recurso representa algo importante do sistema.
- Em um sistema acadêmico, recursos possíveis seriam alunos, cursos, turmas e matrículas.
- Nesta aula, vamos trabalhar apenas com o recurso alunos.

```
Recursos possíveis:

/alunos
/cursos
/turmas
/matriculas
/usuarios
```

Regra didática

A rota deve representar o recurso. A ação deve ser indicada pelo método HTTP.

Endpoint = método HTTP + caminho

O mesmo caminho pode ter ações diferentes dependendo do método

```
GET  /alunos   → listar alunos
POST /alunos   → criar aluno
```

```
GET  /alunos/1 → buscar aluno 1
PUT  /alunos/1 → atualizar aluno 1
```

Método

Indica a intenção da requisição: consultar, criar, atualizar ou remover.

Caminho

Indica o recurso que será acessado.

Endpoint

É a combinação entre método e caminho.

Métodos HTTP essenciais

A ação vem do método, não do nome da rota

Método	Uso principal	Exemplo
GET	Consultar dados	`GET /alunos`
POST	Criar novo recurso	`POST /alunos`
PUT	Atualizar recurso existente	`PUT /alunos/1`
DELETE	Remover ou inativar	`DELETE /alunos/1`

Desenhando uma API REST para alunos

Uma convenção simples

Rota	Operação	
GET /alunos	Listar	Retorna todos os alunos
GET /alunos/:id	Listar	Retorna um aluno específico
POST /alunos	Criar	Cria um novo aluno
PUT /alunos/:id	Atualizar	Atualiza um aluno existente
DELETE /alunos/:id	Deletar	Apaga um aluno.

Observação

Em alguns sistemas, DELETE não apaga fisicamente: apenas inativa o registro para preservar histórico.
.

Estrutura mínima do projeto backend

Organização sugerida para a aula prática

```
aula08-api/
├── package.json
├── tsconfig.json
└── src/
    └── server.ts
```

```
npm init -y
npm install express cors
npm install -D typescript ts-node-dev
@types/express @types/node
```

Scripts úteis

"dev": "ts-node-dev --respawn src/server.ts"

Por enquanto

Sem banco de dados. Dados ficam em memória.

Middleware essencial: express.json()

Sem ele, req.body pode chegar vazio ou indefinido

```
const app = express();

app.use(express.json());
```

O que faz?

Permite que o Express interprete JSON enviado no corpo da requisição.

Onde entra?

Antes das rotas POST e PUT.

Erro comum

Esquecer essa linha e não conseguir ler req.body.

Request e Response

Toda rota recebe uma requisição e envia uma resposta

```
app.get("/alunos", (req: Request, res: Response) =>
{
  return res.status(200).json(alunos);
});
```

req

Representa a requisição recebida.

req.params → dados que vêm no caminho da rota
req.query → dados que vêm depois do ? na URL
req.body → dados que vêm no corpo da requisição

res

Representa a resposta enviada.

res.json()
res.status()
res.send()

return

Evita que o servidor tente responder duas vezes após um erro.

Modelando um aluno em TypeScript

Tipos ajudam a manter o contrato dos dados

```ts
type SituacaoAluno = "Ativo" | "Trancado" | "Formado" | "Em análise" | "Inativo";

type CursoAluno =
  "Engenharia de Software" | "Ciência da Computação" |
  "Sistemas de Informação" | "ADS";

interface Aluno {
  id: number;
  nome: string;
  email: string;
  curso: CursoAluno;
  semestre: number;
  situacao: SituacaoAluno;
}
```

Contrato

Define o formato esperado de um aluno.

Evita bagunça

Reduz inconsistências como “Ativo”, “ativo” e “ATIVO”.

Prévia do banco

Mais tarde, esses campos serão persistidos no Oracle.

Dados em memória: o banco provisório

O array permite focar primeiro na lógica da API

```
let alunos: Aluno[] = [
  {
    id: 1,
    nome: "Ana Clara Martins",
    email: "ana@instituicao.edu.br",
    curso: "Engenharia de Software",
    semestre: 2,
    situacao: "Ativo"
  }
];

let proximoId = 2;
```

Importante

Quando o servidor reinicia, os dados criados são perdidos.

Por que usar agora?

Para entender rotas antes de misturar banco de dados.

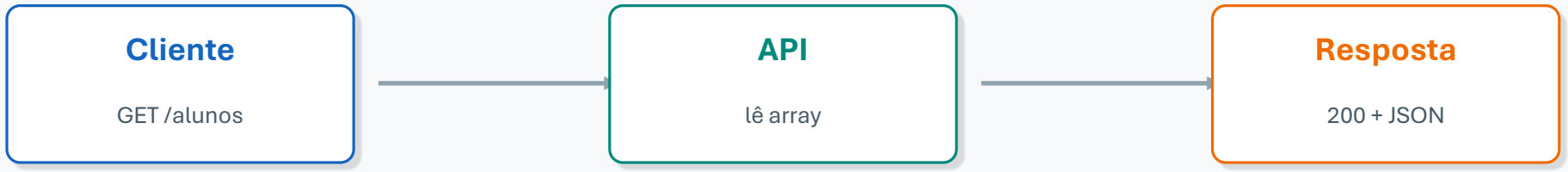
Depois

O Oracle substituirá o array.

Rota GET /alunos

Listar todos os registros do recurso

```
app.get("/alunos", (req: Request, res:
Response) => {
  return res.status(200).json(alunos);
});
```



Rota GET /alunos/:id

Usando parâmetros de rota para buscar um item específico

```
app.get("/alunos/:id", (req: Request, res: Response) => {
  const id = Number(req.params.id);
  const aluno = alunos.find((item) => item.id === id);

  if (!aluno) {
    return res.status(404).json({ mensagem: "Aluno não encontrado." });
  }

  return res.status(200).json(aluno);
});
```

:id

Parte variável da URL.

req.params.id

Valor recebido como texto.

Number(...)

Converte para número.

404

Usado quando não encontrou.

Rota POST /alunos

Criando um novo aluno a partir do corpo da requisição

```
app.post("/alunos", (req: Request, res: Response) => {
  const { nome, email, curso, semestre } = req.body;

  if (!nome || !email || !curso || !semestre) {
    return res.status(400).json({
      mensagem: "Nome, e-mail, curso e semestre são obrigatórios."
    });
  }

  const novoAluno: Aluno = {
    id: proximoId,
    nome, email, curso, semestre,
    situacao: "Ativo"
  };

  alunos.push(novoAluno);
  proximoId++;
  return res.status(201).json(novoAluno);
});
```

req.body

Dados enviados no corpo da requisição.

400 Bad Request

Faltou dado obrigatório.

201 Created

Recurso criado com sucesso.

Status inicial

Todo aluno nasce como Ativo.

Como testar o POST?

A requisição precisa enviar JSON no corpo

```
POST http://localhost:3000/alunos
Content-Type: application/json

{
  "nome": "Bruno Henrique Lima",
  "email": "bruno@instituicao.edu.br",
  "curso": "Ciência da Computação",
  "semestre": 4
}
```

Header

Content-Type informa que o corpo está em JSON.

Body

Contém os dados do novo aluno.

Resposta

API devolve o aluno criada com id e status.

Status HTTP: a API também comunica resultado

Não basta devolver uma mensagem: o código importa

Código	Uso na aula
200 OK	Consulta ou atualização bem-sucedida
201 Created	Aluno criado
400 Bad Request	Dados obrigatórios faltando
404 Not Found	ID não encontrado
500 Internal Server Error	Erro inesperado no servidor

Boa prática: o frontend deve decidir o que fazer com base no status da resposta.

Rota PUT /alunos/:id

Atualizando um aluno existente

```
app.put("/alunos/:id", (req: Request, res: Response) => {
  const id = Number(req.params.id);
  const aluno = alunos.find((item) => item.id === id);

  if (!aluno) {
    return res.status(404).json({ mensagem: "Aluno não encontrado." });
  }

  const { nome, email, curso, semestre, situacao } = req.body;

  if (!nome || !email || !curso || !semestre || !situacao) {
    return res.status(400).json({ mensagem: "Todos os campos são
obrigatórios." });
  }

  aluno.nome = nome;
  aluno.email = email;
  aluno.curso = curso;
  aluno.semestre = semestre;
  aluno.situacao = situacao;

  return res.status(200).json(aluno);
});
```

1. Localiza

Usa o ID da rota para encontrar o aluno.

2. Valida

Verifica se o aluno existe e se os campos vieram.

3. Atualiza

Altera o objeto existente no array.

4. Responde

Retorna 200 + aluno atualizado.

Rota DELETE /alunos/:id

Em um sistema real, apagar pode significar inativar

```
app.delete("/alunos/:id", (req: Request, res: Response) => {
  const id = Number(req.params.id);
  const aluno = alunos.find((item) => item.id === id);

  if (!aluno) {
    return res.status(404).json({ mensagem: "Aluno não encontrado." });
  }

  aluno.situacao = "Inativo";

  return res.status(200).json({
    mensagem: "Aluno inativado com sucesso.",
    aluno
  });
});
```

Exclusão física

Remove definitivamente o registro.

Inativação lógica

Preserva o aluno e muda a situação.

Por que isso importa?

Mantém histórico e rastreabilidade.

Query parameters

Filtros são normalmente passados após o ? na URL

```
GET /alunos?situacao=Ativo
GET /alunos?curso=ADS
```

```
app.get("/alunos", (req: Request, res: Response) => {
  const situacao = req.query.situacao;

  if (situacao) {
    const filtrados = alunos.filter((item) => item.situacao ===
situacao);
    return res.status(200).json(filtrados);
  }

  return res.status(200).json(alunos);
});
```

params

Identifica um recurso.
Ex.: /alunos/1

query

Filtra ou refina uma consulta.
Ex.: ?situacao=Ativo

body

Envia dados maiores no POST/PUT.

Validação básica no backend

A API não deve confiar cegamente no que recebe

- Campos obrigatórios devem ser conferidos.
- IDs inexistentes devem retornar 404.
- Valores inválidos devem retornar 400.
- Toda resposta deve ser clara para quem consome a API.

```
if (!titulo || !descricao || !tipo ||
!prioridade) {
  return res.status(400).json({
    mensagem: "Dados obrigatórios não
informados."
  });
}
```

Observação

Validação no frontend melhora a experiência; validação no backend protege o sistema.

Erros comuns nessa aula

Antecipar problemas economiza tempo na prática

Esquecer express.json()

Sintoma: req.body aparece undefined.

Não converter ID

req.params.id é string; id do objeto costuma ser number.

Responder duas vezes

Falta de return após res.status(...).json(...).

Status errado

Retornar 200 para erro confunde o frontend.

Rota mal desenhada

Usar /criarAluno em vez de POST /alunos.

Testando a API sem frontend

Thunder Client, Insomnia ou Postman simulam o cliente



Testes mínimos

GET /, GET /alunos, GET /alunos/1, POST /alunos, PUT /alunos/1, DELETE /alunos/1

Fluxo mental de uma rota

O padrão se repete em praticamente toda API




```
app.post("/alunos", (req, res) => {
  // 1. recebe a requisição
  // 2. lê req.body
  // 3. valida campos obrigatórios
  // 4. cria e salva o aluno
  // 5. responde com 201 + JSON
});
```

Cheat sheet da Aula

Consulta rápida para os alunos

```
// rota básica
app.get("/alunos", (req, res) => {
  return res.status(200).json(alunos);
});

// parâmetro de rota
const id = Number(req.params.id);

// corpo da requisição
const { nome } = req.body;

// erro
return res.status(404).json({ mensagem: "Não encontrado." });

// criação
return res.status(201).json(novoRegistro);
```

200

Operação bem-sucedida.

201

Registro criado.

400

Dados enviados com problema.

404

Registro não encontrado.

500

Erro inesperado no servidor.